System Design Document

Emergency Stop System

Autonome Fahrzeugplattform – Truck

Kurs: Software Engineering

Gruppe: Die Drei Muskeltiere

DHBW Ravensburg, Campus Friedrichshafen

Datum: März 2026

Bewertungsgewichtung: 75 % Architektur-Dokumentation
25 % Detailed Design

Inhaltsverzeichnis

1 Einleitung **3**
1.1 Scope . . . 3
1.2 Ziel des Dokuments . . . 3
1.3 Abgrenzung Normal Operation vs. Emergency Stop . . . 3
1.4 Dokument-Übersicht . . . 4

2 Gesamtarchitektur des Systems **4**
2.1 Systemkontext . . . 4
2.2 Logische Systemstruktur (Component Overview) . . . 5
2.3 Subsystem-Verantwortlichkeiten und Klassendiagramme . . . 5
2.3.1 Jetson Nano – Klassen . . . 6
2.3.2 Arduino Mega Master – Klassen . . . 6
2.3.3 Arduino Mega Slave – Klassen . . . 7

3 Verteilungsarchitektur (Deployment) **7**
3.1 Komponentenverteilung . . . 8
3.2 Kommunikationskanäle . . . 8
3.3 Rationale der Verteilung . . . 8

4 Dynamisches Verhalten des Systems **9**
4.1 Global Software Control: Auslösung durch Safety-Zone . . . 9
4.2 Boundary Condition: Systemstart (Startup) . . . 10
4.3 Boundary Condition: Freigabe nach Emergency Stop (Release) . . . 11
4.4 Boundary Condition: Freigabe abgelehnt (Release Rejected) . . . 12
4.5 Boundary Condition: Fehlerbehandlung (Error) . . . 12
4.6 Boundary Condition: Shutdown . . . 13

5 Detailed Design: Emergency Stop **13**
5.1 Klassendesign . . . 14
5.1.1 Attribute . . . 14
5.1.2 Öffentliche Operationen (Public Interface) . . . 15
5.1.3 Private Methoden . . . 15
5.1.4 Enum: EStopState . . . 16
5.2 Interner Ablauf (Sequenzdiagramm) . . . 17
5.3 Zustandsautomat (State Machine) . . . 18
5.3.1 Zustandsübergänge . . . 19
5.3.2 Besondere Eigenschaften . . . 19

6 Architektonische Entscheidungen **19**
6.1 Sprachauswahl: C vs. C++ . . . 19
6.2 Verteilte Architektur . . . 20
6.3 Fail-Safe-Prinzip . . . 21
6.4 Klare Verantwortlichkeiten . . . 21
6.5 CAN als Kommunikationsprotokoll . . . 21

7 Traceability: Anforderung → Artefakt **21**

8 Nächste Schritte und offene Punkte **23**

8.1 Priorisierte To-do-Liste 23
8.2 Bekannte Annahmen (zur Überprüfung) 23

Abbildungsverzeichnis

1 Kontextdiagramm – Systemgrenzen und externe Akteure 4
2 Komponentenübersicht – logische Systemstruktur 5
3 Klassendiagramm Jetson Nano: Autonomie-/Wahrnehmungs-Subsystem 6
4 Klassendiagramm Arduino Mega Master: Kommunikations-Gateway . . 6
5 Klassendiagramm Arduino Mega Slave: Sicherheits- und Antriebssubsystem 7
6 Deployment-Diagramm – Hardware/Software-Mapping 8
7 Sequenzdiagramm: Auslösung des Emergency Stop durch Safety-Zone (Rot) 9
8 Sequenzdiagramm: Startup-Verhalten 10
9 Sequenzdiagramm: Manuelle Freigabe nach Emergency Stop 11
10 Sequenzdiagramm: Freigabe abgelehnt (Guard nicht erfüllt) 12
11 Sequenzdiagramm: Fehlerbehandlung (fehlende Sensordaten) 12
12 Sequenzdiagramm: Kontrollierter Shutdown 13
13 Detailliertes Klassendiagramm – EmergencyStop inkl. private Methoden 14
14 Internes Sequenzdiagramm – Ablauf innerhalb von EmergencyStop . . 17
15 Zustandsautomat – vollständiges Verhalten von EmergencyStop 18

1 Einleitung

Dieses Dokument beschreibt das **Program Design** des verteilten Not-Halt-Systems einer autonomen Fahrzeugplattform (Truck). Es hat den Charakter eines *System Design Documents* (SDD) und dokumentiert die Softwarearchitektur sowie das detaillierte Design des Teilsystems **Emergency Stop**.

1.1 Scope

- **In Scope:** Software im Truck (Jetson Nano, Arduino Mega Master, Arduino Mega Slave), Architektur, Kommunikationsschnittstellen, Emergency-Stop-Logik, Boundary Conditions.
- **Out of Scope:** PC-Anwendung, ROS-Integration, WLAN-Kommunikation, Details der Sensor-Fusion (wird als Blackbox behandelt).
- **Vereinfachungen:** LiDAR liefert direkt die Distanz zum nächstliegenden Objekt; Sensor-Fusion ist vorgelagert und für dieses Dokument transparent.

1.2 Ziel des Dokuments

Das SDD richtet sich an Entwicklerinnen und Entwickler, die

1. die Gesamtarchitektur und die Subsystem-Verantwortlichkeiten verstehen,
2. das Deployment auf den Hardwareeinheiten nachvollziehen,
3. das dynamische Verhalten (Sequenzen, Boundary Conditions) analysieren und
4. das Emergency-Stop-Modul detailliert implementieren wollen.

1.3 Abgrenzung Normal Operation vs. Emergency Stop

Das System unterscheidet zwei Betriebsmodi:

Normal Operation (Gelb/Grün) Der *TTC-Observer* (Time-To-Collision) überwacht die Umgebung und erlaubt eine automatische Reaktion auf Hindernisse. Der Fahrvorgang wird angepasst, das Fahrzeug kann aber weiterfahren. TTC ist **kein** Bestandteil des Emergency-Stop-Moduls.

Safety Zone / Emergency Stop (Rot) Sobald ein Objekt die rote Sicherheitszone unterschreitet, ein Pilzknopf gedrückt oder ein Fronttaster ausgelöst wird, übernimmt der *Emergency Stop*. Das Fahrzeug bleibt stehen, bis eine **manuelle Freigabe** erfolgt (Guard-Bedingung: Safety-Zone nicht mehr aktiv, kein Fehler).

1.4 Dokument-Übersicht

Abschnitt	Inhalt
§ 2	Systemkontext, Subsystem-Zerlegung, Klassendiagramme
§ 3	Hardware/Software-Mapping (Deployment)
§ 4	Global Software Control + Boundary Conditions (Sequenzen)
§ 5	Detailed Design Emergency Stop
§ 6	Architektonische Entscheidungen (inkl. Sprachauswahl)
§ 7	Traceability: Anforderung → Artefakt
§ 8	Nächste Schritte / offene Punkte

2 Gesamtarchitektur des Systems

2.1 Systemkontext

Das Fahrzeug besteht aus drei physisch getrennten Recheneinheiten, die über definierte Kommunikationskanäle miteinander verbunden sind:

- **Jetson Nano** – Hochleistungs-Recheneinheit für Wahrnehmung und autonome Logik.
- **Arduino Mega Master** – Kommunikations-Gateway zwischen Jetson und Slave.
- **Arduino Mega Slave** – Sicherheitskritische Einheit nahe der Motorsteuerung; hier läuft der Emergency Stop.

Die Kommunikation erfolgt über:

- **LAN** (Jetson Nano ↔ Arduino Mega Master)
- **CAN-Bus** (Arduino Mega Master ↔ Arduino Mega Slave)

Die Platzierung des Emergency Stop auf dem Slave minimiert Kommunikationsverzögerungen zur Motorsteuerung (*DriveControlUnit*) und stellt eine deterministische Reaktionszeit sicher.

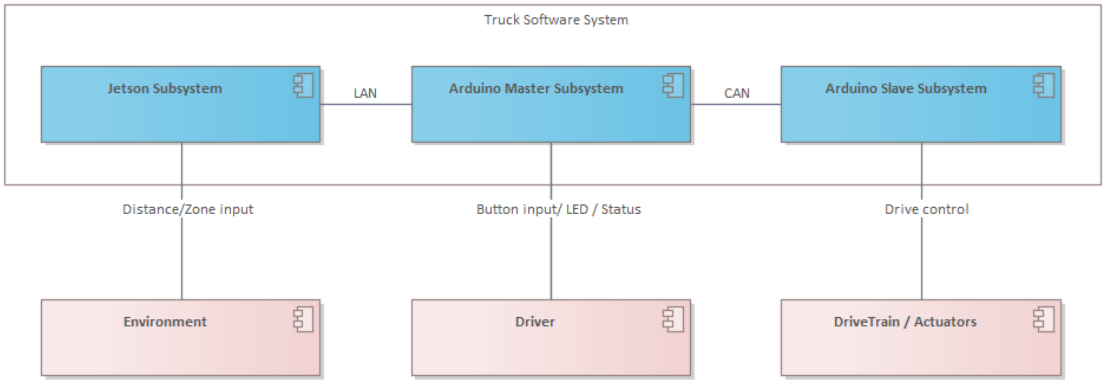

Abbildung 1: Kontextdiagramm – Systemgrenzen und externe Akteure

2.2 Logische Systemstruktur (Component Overview)

Die logische Architektur zeigt die funktionale Zerlegung des Systems in Subsysteme. Abbildung 2 gibt einen Überblick.

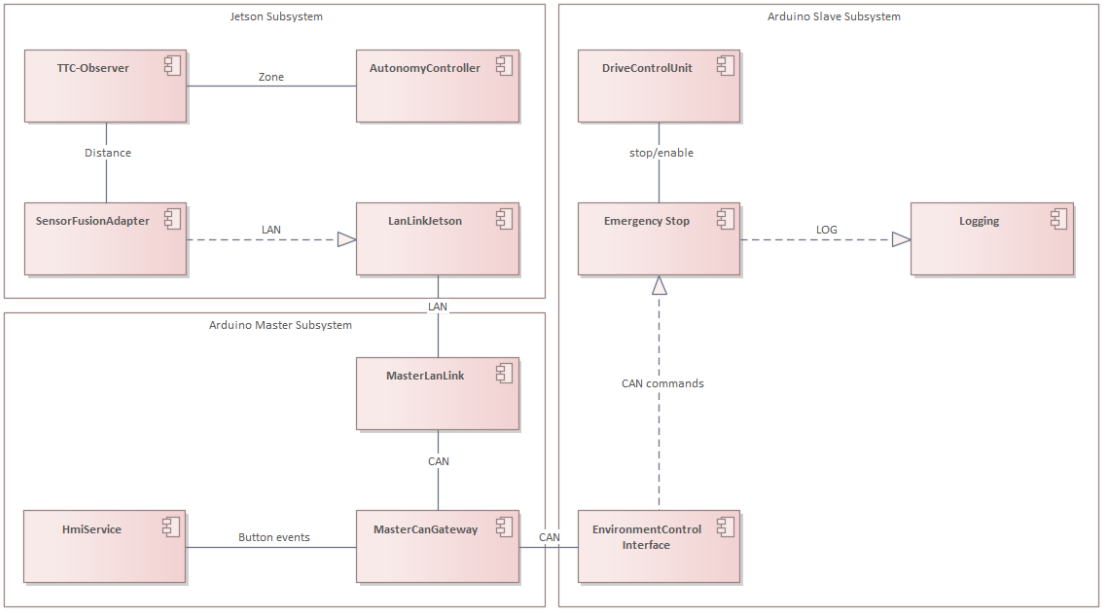

Abbildung 2: Komponentenübersicht – logische Systemstruktur

Wesentliche Eigenschaften der Architektur:

- Der **TTC-Observer** verarbeitet Sensordaten zur Bestimmung der Fahrzone (Grün/Gelb) und gehört ausschließlich zur *Normal Operation*.
- Der **Emergency Stop** verarbeitet ausschließlich CAN-basierte Kommandos und ist von der Autonomie-Logik entkoppelt.
- Sensorik wird über das `EnvironmentControl`-Interface in CAN-Nachrichten übersetzt (Sensor-Fusion als Blackbox).
- Das **Logging**-Modul ist als separates Querschnittsmodul modelliert; es wird von Emergency Stop bei jeder Zustandsänderung aufgerufen.
- Eine direkte Abhängigkeit zwischen TTC-Observer und Emergency Stop **existiert nicht** – dies sichert die Unabhängigkeit der Sicherheitsfunktion.

2.3 Subsystem-Verantwortlichkeiten und Klassendiagramme

Die folgenden Klassendiagramme zeigen Attribute und öffentliche Operationen pro Hardware-Subsystem.

2.3.1 Jetson Nano – Klassen

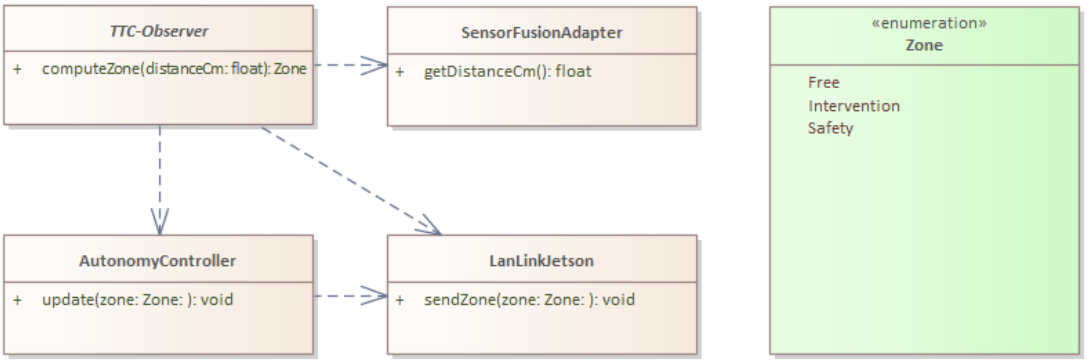

Abbildung 3: Klassendiagramm Jetson Nano: Autonomie-/Wahrnehmungs-Subsystem

Hauptverantwortlichkeiten des Jetson-Subsystems:

`AutonomyController` Koordiniert autonomes Fahrverhalten; empfängt Zonenstatus vom TTC-Observer.

`TTCObserver` Berechnet Time-To-Collision aus Sensordaten; sendet ZoneUpdate-Kommandos (Normal/Warnung). Zuständig für Gelb- und Grün-Zonen.

`SensorFusionAdapter` Abstrahiert die Sensor-Fusion (Kamera + LiDAR); liefert normierte Distanz-/Objektdaten. Intern als Blackbox behandelt.

2.3.2 Arduino Mega Master – Klassen

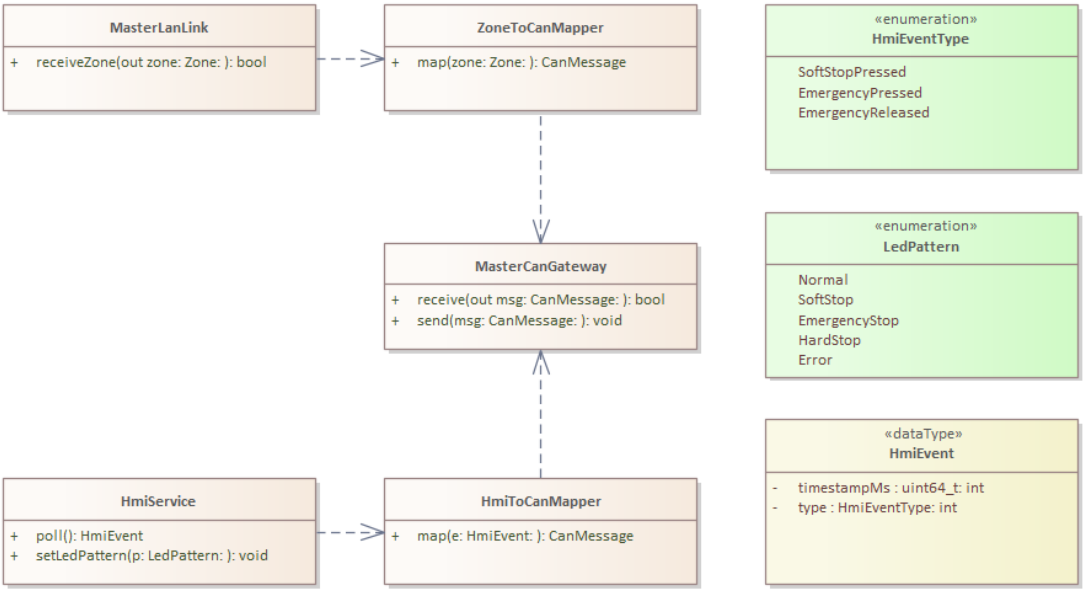

Abbildung 4: Klassendiagramm Arduino Mega Master: Kommunikations-Gateway

Hauptverantwortlichkeiten des Master-Subsystems:

`MasterLanLink` Empfängt Befehle vom Jetson über LAN und leitet sie auf den CAN-Bus weiter.

`MasterCanGateway` Kapselt CAN-Kommunikation; wandelt LAN-Nachrichten in CAN-Pakete um. *[Annahme: CAN-Datenklasse als eigene Klasse modelliert (OOP-Anforderung).]*

`HmiService` Verarbeitet Signale des Pilzknopfs (Emergency-Button) und des Fronttasters; erzeugt entsprechende CAN-Ereignisse.

2.3.3 Arduino Mega Slave – Klassen

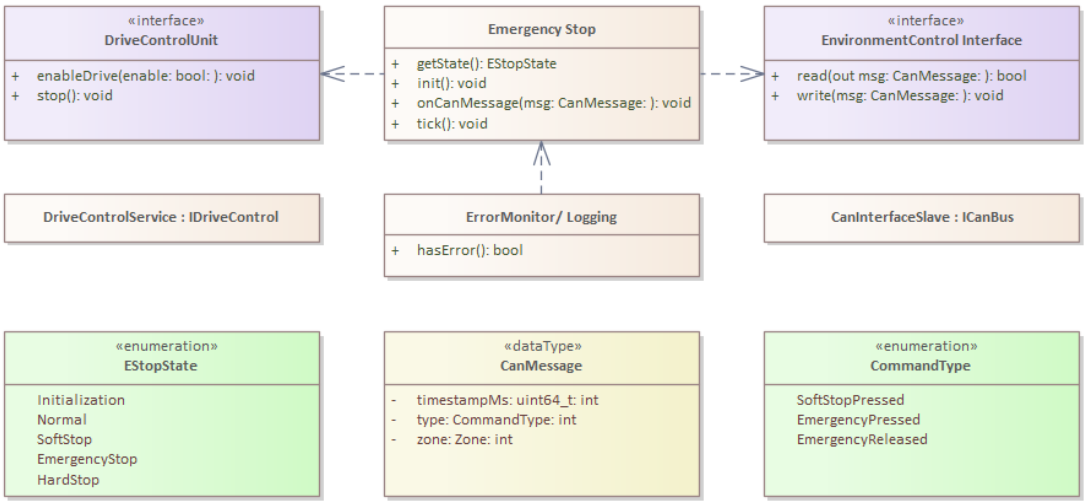

Abbildung 5: Klassendiagramm Arduino Mega Slave: Sicherheits- und Antriebssubsystem

Hauptverantwortlichkeiten des Slave-Subsystems:

`EnvironmentControl` Schnittstelle zur Empfangsseite des CAN-Busses; übersetzt eingehende Pakete in interne Ereignisse für den Emergency Stop.

`EmergencyStop` Zentrale Sicherheitsklasse (Detailed Design in Abschnitt 5). Steuert den Zustandsautomaten und die DriveControlUnit.

`DriveControlUnit` Abstrahiert die Motorsteuerung; empfängt ausschließlich Stop-/Freigabe-Befehle vom Emergency Stop.

`Logger` Querschnittsmodul für Error-Logging; unterstützt Level (`ERROR`, `WARNING`, `INFO`) mit Zeitstempel.

3 Verteilungsarchitektur (Deployment)

Das Deployment-Diagramm (Abbildung 6) ordnet Softwarekomponenten den physischen Hardwareeinheiten zu und macht die Kommunikationskanäle sichtbar.

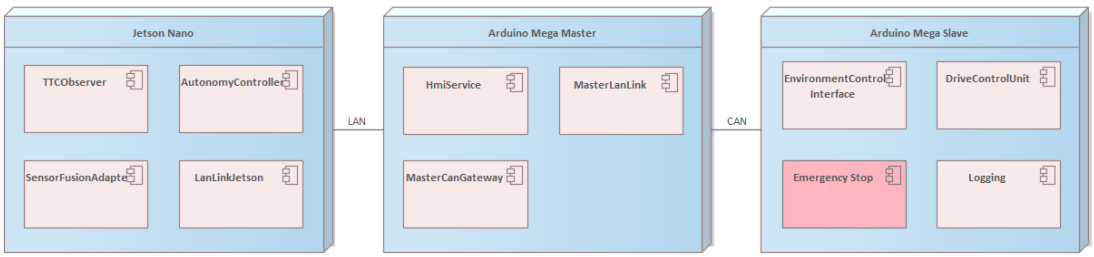

Abbildung 6: Deployment-Diagramm – Hardware/Software-Mapping

3.1 Komponentenverteilung

Hardware-Knoten	Softwarekomponenten
Jetson Nano	`AutonomyController`, `TTCObserver`, `SensorFusionAdapter`
Arduino Mega Master	`MasterLanLink`, `MasterCanGateway`, `HmiService`
Arduino Mega Slave	`EnvironmentControl`, `EmergencyStop`, `DriveControlUnit`, `Logger`

3.2 Kommunikationskanäle

- **LAN** (Ethernet): Jetson Nano ↔ Arduino Mega Master.
  Überträgt Autonomie-Kommandos und Zonen-Updates vom Jetson zum Master.
- **CAN-Bus**: Arduino Mega Master ↔ Arduino Mega Slave.
  Überträgt CAN-Pakete mit Zonenstatus, Emergency-Ereignissen und Steuerbefehlen. *[Annahme: CAN-Datenklasse kapselt Paketstruktur (ID, Payload, Timestamp).]*

3.3 Rationale der Verteilung

Die Platzierung des Emergency Stop direkt auf dem Arduino Mega Slave neben der `DriveControlUnit` verfolgt folgende Ziele:

1. **Minimale Latenz:** Kein zusätzlicher Kommunikationsweg zwischen Sicherheitslogik und Motorsteuerung.
2. **Isolation:** Ausfall des Jetson oder des Masters hat keinen direkten Einfluss auf die Fähigkeit, einen Stopp einzuleiten.
3. **Determinismus:** Der Slave führt eine einfache Echtzeit-Logik aus; kein OS-Overhead wie auf dem Jetson.

TODO: Latenzzahlen (typische CAN-Zykluszeit, LAN-Latenz) ergänzen, sobald Messungen vorliegen.

4 Dynamisches Verhalten des Systems

Dieser Abschnitt beschreibt das Laufzeitverhalten anhand von Sequenzdiagrammen. Er umfasst sowohl den *Global Software Control* (regulärer Betrieb mit Emergency Stop) als auch die *Boundary Conditions* (Systemstart, Shutdown, Fehlerbehandlung).

4.1 Global Software Control: Auslösung durch Safety-Zone

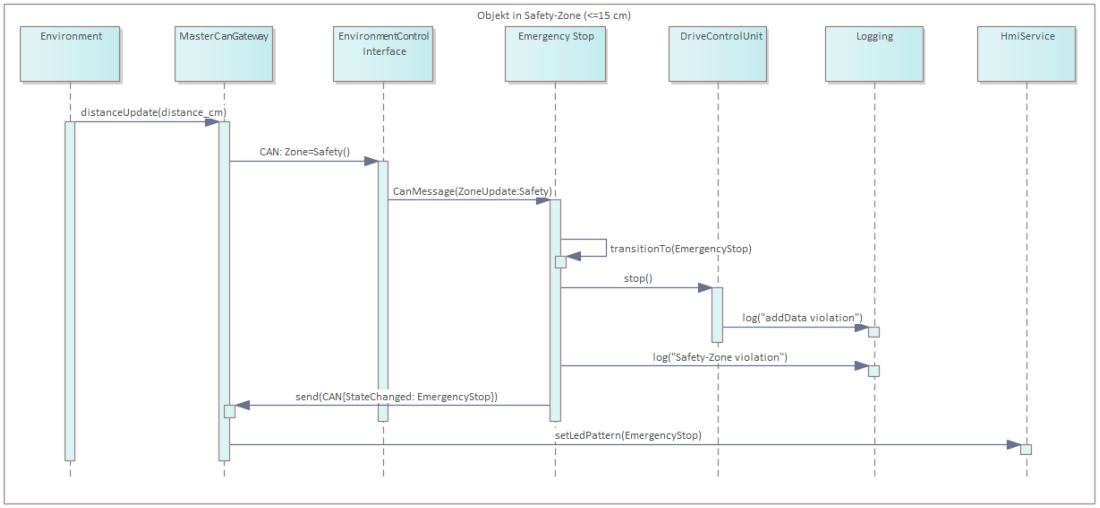

Abbildung 7: Sequenzdiagramm: Auslösung des Emergency Stop durch Safety-Zone (Rot)

Wird eine Distanz $\leq 15\,\text{cm}$ erkannt (rote Sicherheitszone), läuft folgender Ablauf ab:

1. Der `SensorFusionAdapter` auf dem Jetson meldet ein Objekt in der Safety-Zone.
2. Der `TTCObserver` erzeugt ein `ZoneUpdate(Safety)`-Ereignis.
3. Über LAN → `MasterLanLink` → `MasterCanGateway` wird ein CAN-Paket generiert.
4. Das CAN-Paket wird vom `EnvironmentControl`-Interface auf dem Slave empfangen und an `EmergencyStop` weitergegeben.
5. Der Emergency Stop wechselt in den Zustand `EmergencyStop`, sendet einen **Stop-Befehl** an die `DriveControlUnit` und löst einen `Logger.log(ERROR, SSafety zone triggered")`-Aufruf aus.
6. Die Status-LED wechselt auf Rot.

Wichtig: Das Fahrzeug bleibt blockiert, bis eine manuelle Freigabe die Guard-Bedingung erfüllt (vgl. Abschnitt 4.3).

4.2 Boundary Condition: Systemstart (Startup)

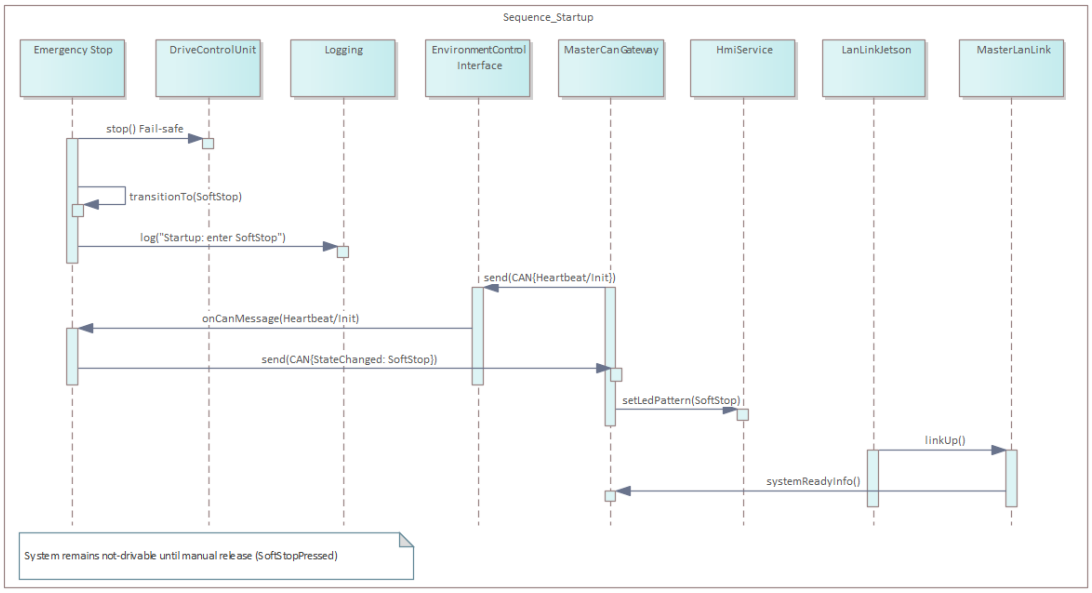

Abbildung 8: Sequenzdiagramm: Startup-Verhalten

Beim Systemstart werden alle Komponenten initialisiert. Der Emergency Stop beginnt automatisch im Zustand `SoftStop` (*Safe-Default*): Das Fahrzeug ist erst nach einer expliziten manuellen Freigabe fahrbereit.

Ablauf:

1. Alle Hardware-Einheiten starten ihre Initialisierungsroutine.
2. `EmergencyStop` wechselt von `Initialization` nach `SoftStop`.
3. `Logger` registriert `INFO: System startup complete`.
4. Fahrzeug wartet auf Freigabe-Signal (Taster/Button).

4.3 Boundary Condition: Freigabe nach Emergency Stop (Release)

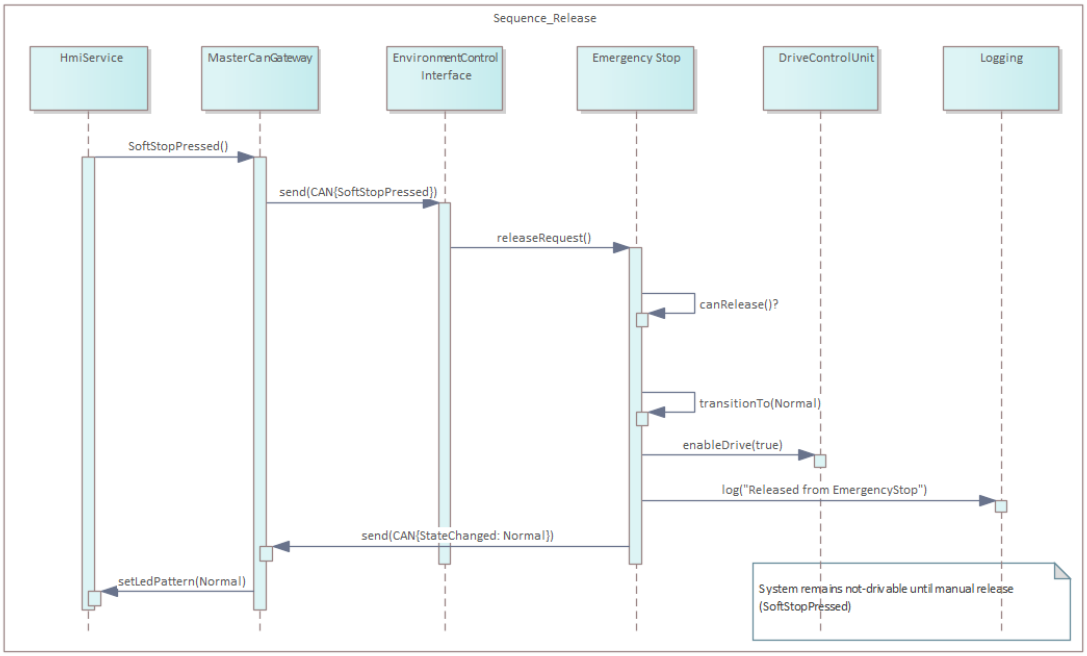

Abbildung 9: Sequenzdiagramm: Manuelle Freigabe nach Emergency Stop

Nach einem Emergency Stop ist eine manuelle Freigabe erforderlich. Die Guard-Bedingung muss vollständig erfüllt sein:

- Safety-Zone ist **nicht** mehr aktiv (kein Objekt in Rot).
- Kein Fehler ist aktiv (`errorFlag == false`).

Erst dann erfolgt:

1. Zustandswechsel `EmergencyStop` → `Normal`.
2. Aktivierung der `DriveControlUnit` (Freigabe der Motorsteuerung).
3. `Logger.log(INFO, SSystem released, resuming normal operation")`.

4.4 Boundary Condition: Freigabe abgelehnt (Release Rejected)

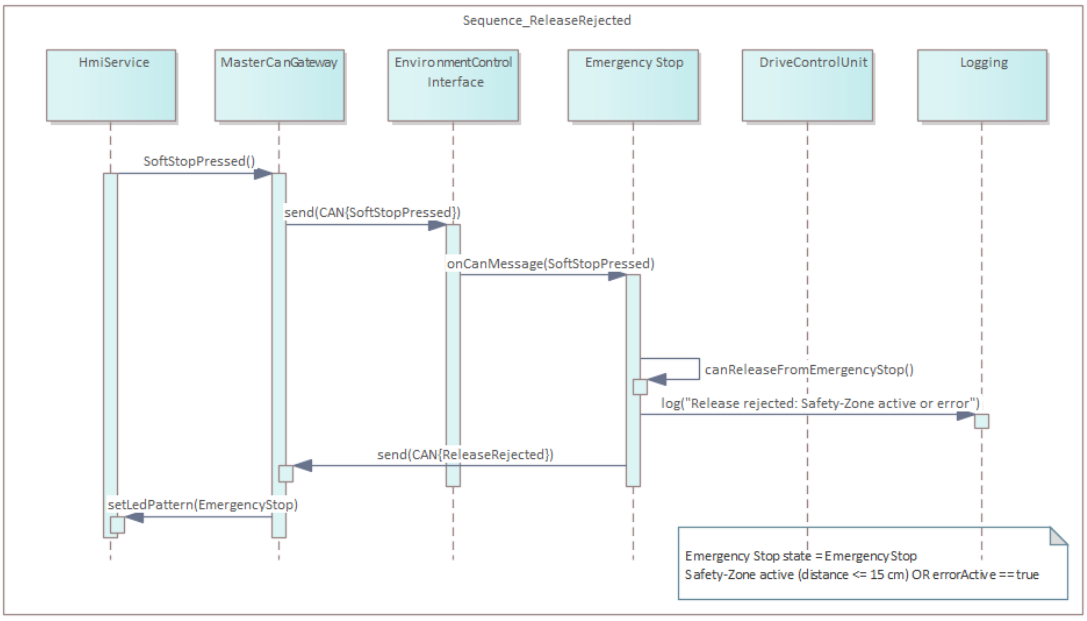

Abbildung 10: Sequenzdiagramm: Freigabe abgelehnt (Guard nicht erfüllt)

Ist die Safety-Zone noch aktiv oder liegt ein Fehler vor, wird der Freigabe-Request **abgelehnt**. Der Zustand bleibt unverändert (`EmergencyStop`). `Logger` protokolliert `WARNING: Release rejected - condition not met`.

4.5 Boundary Condition: Fehlerbehandlung (Error)

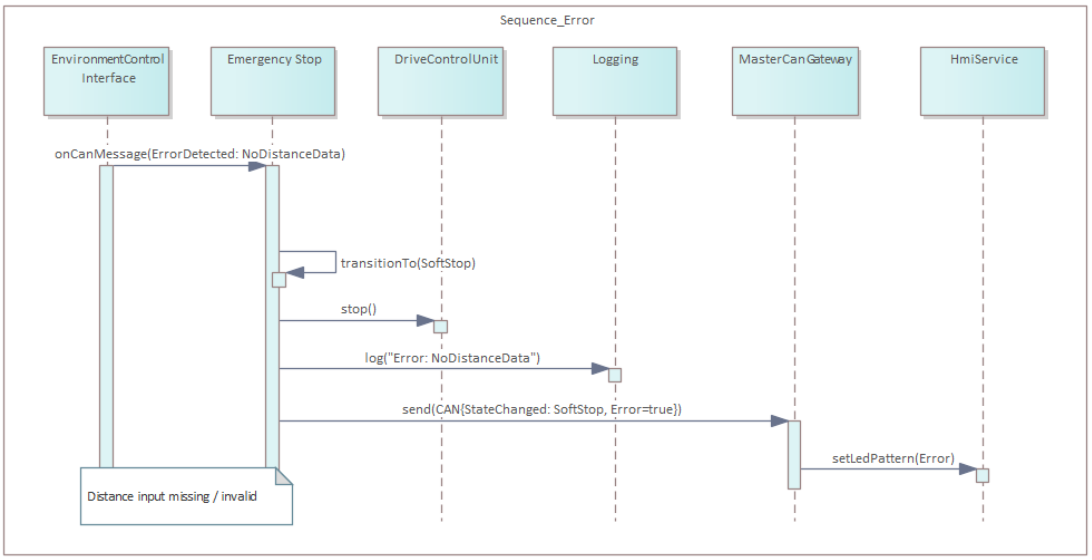

Abbildung 11: Sequenzdiagramm: Fehlerbehandlung (fehlende Sensordaten)

Fehlen gültige Sensordaten (z. B. LiDAR-Ausfall, CAN-Timeout), greift das **Fail-Safe-Prinzip**:

1. `EnvironmentControl` detektiert fehlende/ungültige CAN-Daten.
2. `EmergencyStop` erhält `ErrorDetected`-Ereignis.
3. Zustandswechsel nach `SoftStop` (falls noch nicht gestoppt).
4. `Logger.log(ERROR, SSensor data missing - soft stop engaged")`.

Regel: Keine gültigen Daten ⇒ Stopp + Logging (keine Ausnahmen).

4.6 Boundary Condition: Shutdown

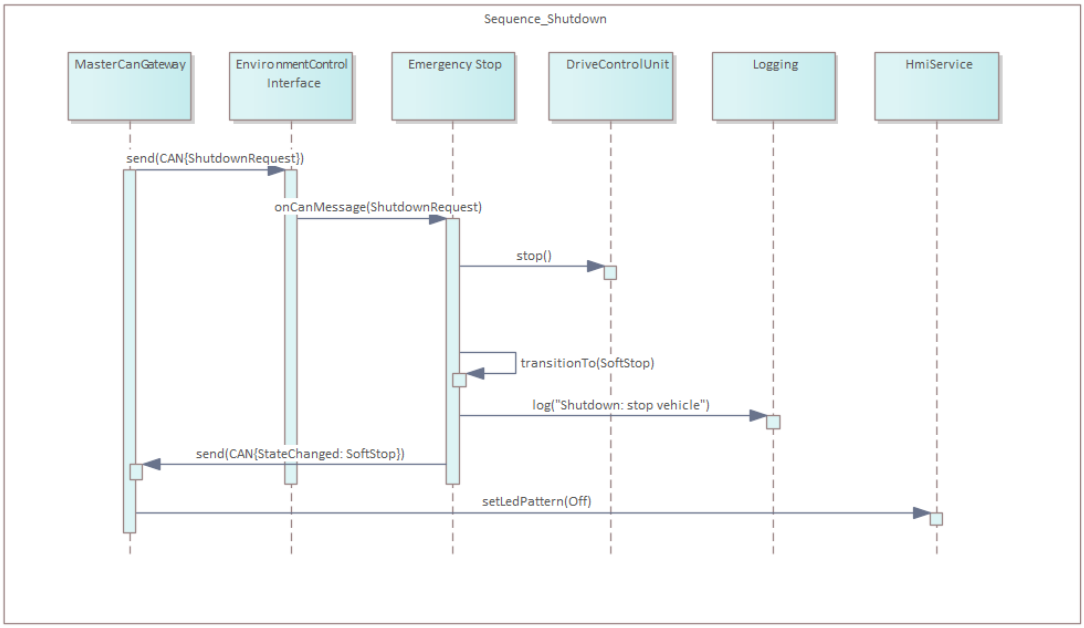

Abbildung 12: Sequenzdiagramm: Kontrollierter Shutdown

Bei einem `ShutdownRequest`:

1. `EmergencyStop` empfängt das Shutdown-Ereignis.
2. Zustandswechsel nach `SoftStop`.
3. Stop-Befehl an `DriveControlUnit`.
4. `Logger.log(INFO, SShutdown initiated")`.
5. Alle Subsysteme fahren geordnet herunter.

5 Detailed Design: Emergency Stop

Dieser Abschnitt fokussiert ausschließlich auf das `EmergencyStop`-Modul und beschreibt sein Klassendesign, den internen Ablauf sowie den vollständigen Zustandsautomaten.

5.1 Klassendesign

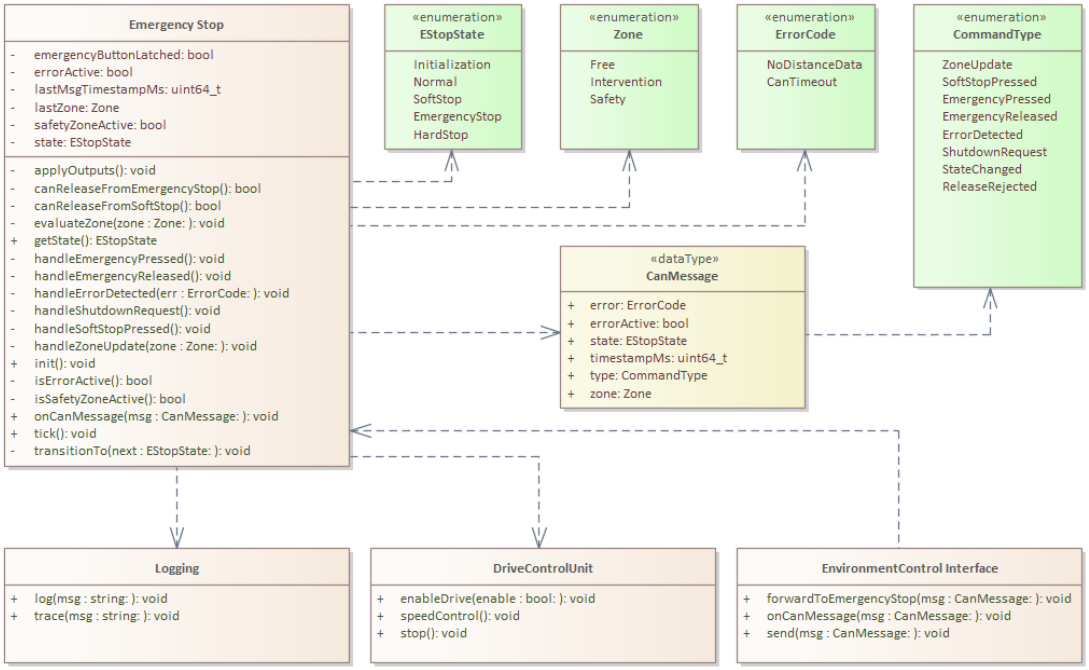

Abbildung 13: Detailliertes Klassendiagramm – EmergencyStop inkl. private Methoden

Die Klasse `EmergencyStop` kapselt den vollständigen Sicherheitszustand.

5.1.1 Attribute

Name	Typ	Beschreibung
`currentState`	`EStopState`	Aktueller Zustand des Automaten
`errorFlag`	`bool`	Aktiver Fehler vorhanden
`safetyZoneActive`	`bool`	Safety-Zone (Rot) gerade aktiv
`driveRef`	`DriveControlUnit*`	Referenz auf Motorsteuerung
`loggerRef`	`Logger*`	Referenz auf Logging-Modul

5.1.2 Öffentliche Operationen (Public Interface)

Signatur	Beschreibung
`onZoneUpdate(zone: Zone) : void`	Eingehende Zonenänderung verarbeiten (Safety / Normal).
`onEmergencyButton() : void`	Pilzknopf-Event verarbeiten.
`onFrontTrigger() : void`	Fronttaster-Event verarbeiten.
`onReleaseRequest() : void`	Manuelle Freigabe anfordern; Guard wird geprüft.
`onErrorDetected() : void`	Fehler-Ereignis (z. B. fehlende Sensordaten) verarbeiten.
`onShutdownRequest() : void`	Kontrollierter Shutdown einleiten.

5.1.3 Private Methoden

Alle internen Handler sind als **private Methoden** implementiert, um die Kapselung der Zustandslogik zu gewährleisten.

Signatur	Beschreibung
`triggerEmergencyStop() : void`	Setzt Zustand auf `EmergencyStop`; ruft `stopDrive()` auf.
`triggerSoftStop() : void`	Setzt Zustand auf `SoftStop`; ruft `stopDrive()` auf.
`stopDrive() : void`	Sendet Stop-Befehl an `DriveControlUnit`.
`enableDrive() : void`	Sendet Freigabe-Befehl an `DriveControlUnit`.
`checkReleaseGuard() : bool`	Prüft: `!safetyZoneActive && !errorFlag`.
`logEvent(level: LogLevel, msg: string) : void`	Delegiert an `Logger`; fügt Zeitstempel hinzu.
`transitionTo(state: EStopState) : void`	Führt Zustandsübergang durch und protokolliert ihn.

5.1.4 Enum: EStopState

Wert	Bedeutung
`Initialization`	Startphase; alle Aktuatoren gesperrt.
`SoftStop`	Sicherer Stopp; automatisch wiederherstellbar nach Freigabe.
`Normal`	Normalbetrieb; `DriveControlUnit` freigegeben.
`EmergencyStop`	Notfall-Stopp; blockiert bis manuelle Freigabe + Guard-Check.
`HardStop`	Höchste Priorität; hardware-seitiger Stopp (kein SW-Override).

TODO: Vollständige Liste aller privaten Methoden mit Vor-/Nachbedingungen (Design by Contract) noch auszuarbeiten.

5.2 Interner Ablauf (Sequenzdiagramm)

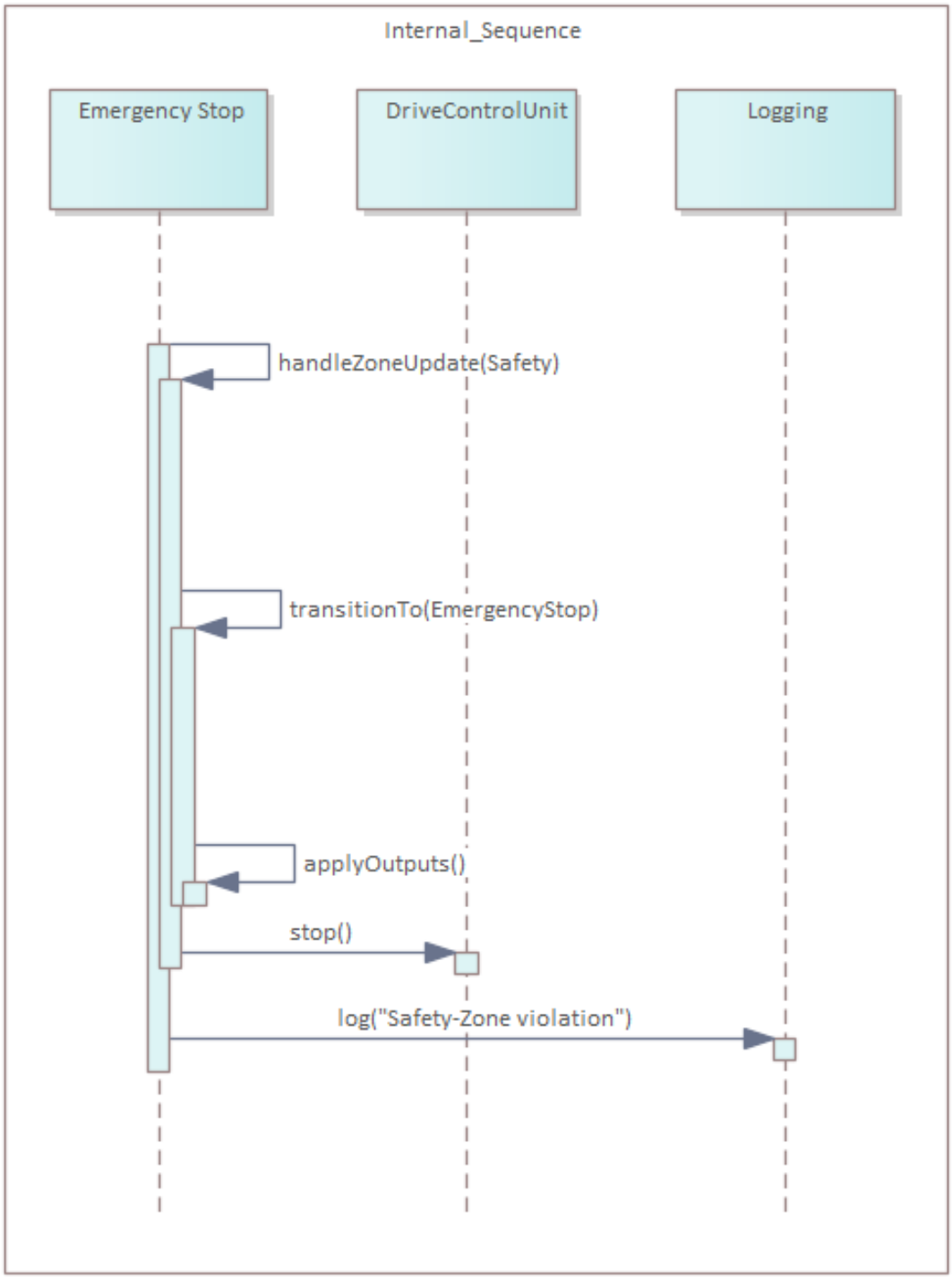

Abbildung 14: Internes Sequenzdiagramm – Ablauf innerhalb von EmergencyStop

Das interne Sequenzdiagramm (Abbildung 14) zeigt, wie ein eingehender Event durch die privaten Methoden verarbeitet wird. Die Verarbeitung folgt immer dem gleichen Schema:

1. **Event-Empfang** über eine öffentliche Methode (z. B. `onZoneUpdate()`).

2. **Guard-Prüfung** (falls erforderlich) durch `checkReleaseGuard()`.
3. **Zustandsübergang** über `transitionTo()` mit automatischem Logging.
4. **Aktorsteuerung**: Aufruf von `stopDrive()` oder `enableDrive()`.
5. **Logging** über `logEvent()` mit Level und Zeitstempel.

Diese Trennung hält die Event-Handler frei von direkten Hardware-Zugriffen und ermöglicht einfaches Unit-Testing.

5.3 Zustandsautomat (State Machine)

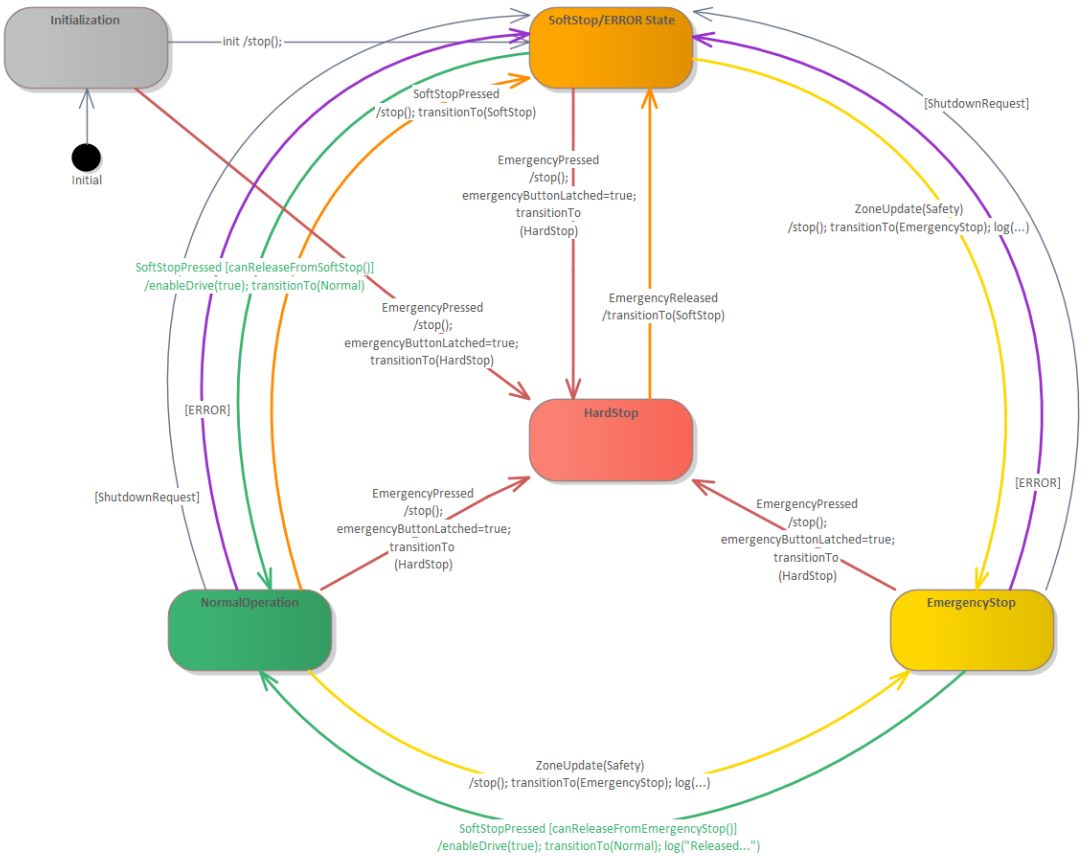

Abbildung 15: Zustandsautomat – vollständiges Verhalten von EmergencyStop

Der Zustandsautomat (Abbildung 15) definiert das vollständige Verhalten von `EmergencyStop`.

5.3.1 Zustandsübergänge

Von	Trigger / Guard	Nach
`Initialization`	SystemReady	`SoftStop`
`SoftStop`	ReleaseRequest [Guard OK]	`Normal`
`Normal`	ZoneUpdate(Safety)	`EmergencyStop`
`Normal`	EmergencyButton / FrontTrigger	`EmergencyStop`
`Normal`	ErrorDetected	`SoftStop`
`Normal`	ShutdownRequest	`SoftStop`
`EmergencyStop`	ReleaseRequest [Guard OK]	`Normal`
`EmergencyStop`	ReleaseRequest [Guard FAIL]	`EmergencyStop`
`EmergencyStop`	ErrorDetected	`SoftStop`
`SoftStop`	ErrorDetected	`SoftStop`
`* (alle)`	HardStopSignal	`HardStop`

5.3.2 Besondere Eigenschaften

- **HardStop** hat die höchste Priorität; er kann aus jedem Zustand erreicht werden und ist nicht softwareseitig aufhebbar.
- **EmergencyStop** erfordert zwingend eine manuelle Freigabe plus erfolgreiche Guard-Prüfung.
- **ErrorDetected** führt immer zu `SoftStop` (Fail-Safe-Prinzip).
- **ShutdownRequest** führt immer zu `SoftStop`.

TODO: Activity-Diagramme für die wichtigsten privaten Methoden (`checkReleaseGuard`, `triggerEmergencyStop`) ausarbeiten oder begründen, warum die State-Machine hinreichend ist.

6 Architektonische Entscheidungen

6.1 Sprachauswahl: C vs. C++

	C	C++
Vorteile	Volle Kontrolle, minimaler Overhead, weit verbreitet auf Embedded	
	OOP (Klassen, Vererbung), STL, bessere Kapselung, Lesbarkeit	
Nachteile	Keine Objektorientierung, schlechte Kapselung, fehleranfälliger Code	
	Leicht erhöhter Code-Footprint, RTTI/Exceptions ggf. deaktivieren	
Eignung Arduino Mega	Ja	Ja (Arduino-Framework basiert auf C++)
Eignung Jetson Nano	Ja	Ja

Entscheidung: C++

Begründung:

- Das Modell ist objektorientiert (Klassen, Interfaces, Enums). C++ erlaubt eine direkte 1:1-Umsetzung des UML-Designs.
- Arduino-Framework basiert nativ auf C++ (Klassen wie `HardwareSerial` sind C++-Klassen).
- Exceptions und RTTI werden **deaktiviert** (Arduino-Standard), um Overhead zu vermeiden.
- Die `Logger`-Klasse, `DriveControlUnit`-Abstraktion und das Interface `EnvironmentControl` profitieren von OOP-Kapselung.

Alternativen:

- *Reines C* wäre möglich, würde aber alle Klassen als Structs mit Funktionszeigern erfordern – erheblich mehr Boilerplate ohne Mehrwert.
- *Rust* wäre sicherheitstechnisch attraktiv, ist aber auf Arduino- Targets nicht offiziell unterstützt.

TODO: Kurze Begründung der Sprachauswahl (1–2 Sätze) ins finale Dokument aufnehmen und mit dem Team abstimmen.

6.2 Verteilte Architektur

Entscheidung: Drei getrennte Recheneinheiten (Jetson, Master, Slave) statt Mono-Architektur.

Rationale:

- *Separation of Concerns:* Wahrnehmung (Jetson) und Sicherheit (Slave) sind physisch und softwareseitig getrennt.
- *Determinismus:* Der Slave führt eine einfache Regellogik ohne OS-Overhead aus.
- *Fault Isolation:* Ein Absturz des Jetson deaktiviert den Emergency Stop nicht (Fail-Safe: Slave stoppt selbstständig bei ausbleibenden Daten).

Alternative: Alles auf dem Jetson Nano (Mono-Architektur). Wurde verworfen, da dies einen Single Point of Failure schafft und die Reaktionszeit durch OS-Scheduling unvorhersehbar wird.

6.3 Fail-Safe-Prinzip

Entscheidung: Im Zweifel immer stoppen.

Konkrete Umsetzungen:

- Startup beginnt in `SoftStop` (kein automatisches Losfahren).
- Fehlende Sensordaten ⇒ `SoftStop` + Logging.
- `HardStop` ist von keiner Zustand aus aufhebbar.

6.4 Klare Verantwortlichkeiten

Komponente	Alleinige Verantwortlichkeit
TTCObserver	Normal Operation (Gelb/Grün); kein Zugriff auf Emergency Stop.
EmergencyStop	Sicherheits-Zustände (Rot, Knopf, Taster); kein Zugriff auf TTC.
Logger	Persistentes Logging; keine Entscheidungslogik.
DriveControlUnit	Motorsteuerung; reagiert nur auf explizite Stop-/Go-Befehle.

6.5 CAN als Kommunikationsprotokoll

Rationale: CAN ist für Automotive-Systeme optimiert, bietet priorisierungsbasierte Arbitrierung und eine hardware-gesicherte Fehlerkennung (CRC, ACK). Dies passt besser zu den Echtzeit-Anforderungen einer Sicherheitsfunktion als eine LAN-basierte Lösung.

[Annahme: CAN-Zykluszeit $< 5\,\mathrm{ms}$; konkrete Werte aus Systemtest nachzutragen.]

7 Traceability: Anforderung → Artefakt

Die folgende Tabelle ordnet jede Pflichtanforderung der Aufgabenstellung einem konkreten Artefakt in diesem Dokument zu.

Anforderung (Aufgabenstellung)	**Artefakt / Diagramm**	**Abschnitt**
Architektur-Übersicht (Context Diagram)	Abb. 1 01_Context.png	§ 2.1
Subsystem-Zerlegung + Verantwortlichkeiten	Abb. 2 02_Component_Overview.png	§ 2.2
Class Diagrams (Jetson)	Abb. 3 02_Jetson_ClassDiagram.png	§ 2.3
Class Diagrams (Master)	Abb. 4 02_Master_ClassDiagram.png	§ 2.3
Class Diagrams (Slave)	Abb. 5 02_Slave_ClassDiagram.png	§ 2.3
Hardware/Software-Mapping (Deployment)	Abb. 6 03_Deployment.png	§ 3
Global Software Control (Sequenz Safety-Zone)	Abb. 7 04_Sequence_SafetyZone.png	§ 4.1
Boundary Condition: Startup	Abb. 8 05_Sequence_Startup.png	§ 4.2
Boundary Condition: Shutdown	Abb. 12 05_Sequence_Shutdown.png	§ 4.6
Boundary Condition: Error	Abb. 11 05_Sequence_Error.png	§ 4.5
Boundary Condition: Release	Abb. 9 05_Sequence_Release.png	§ 4.3
Boundary Condition: Release Rejected	Abb. 10 05_Sequence_ReleaseRejected.png	§ 4.4
Detailed Design ES: Klasse + private Methoden	Abb. 13 06_EStop_Class_Detailed.png + Tabellen	§ 5.1
Detailed Design ES: Internes Sequenzdiagramm	Abb. 14 06_EStop_Internal_Sequence.png	§ 5.2
Detailed Design ES: State Machine	Abb. 15 06_EStop_StateMachine.png	§ 5.3
Sprachauswahl (C vs. C++) + Rationale	Text + Tabelle	§ 6.1
Architektur-Alternativen + Rationale	Text	§ 6.2
Activity-Diagramme pro private Methode	**[TODO: Ausstehend]**	§ 8

8 Nächste Schritte und offene Punkte

Die folgende Liste fasst die wichtigsten offenen Punkte zusammen, priorisiert nach Relevanz für die Bewertung.

8.1 Priorisierte To-do-Liste

1. **[P1 – Detailed Design] Activity-Diagramme für private Methoden ausarbeiten.**
   Mindestens für `checkReleaseGuard()` und `triggerEmergencyStop()` ein Activity-Diagramm erstellen *oder* schriftlich begründen, warum der Zustandsautomat (Abschnitt 5.3) als alleinige dynamische Modellierung ausreicht. **[TODO: Activity-Diagramme erstellen oder Begründung formulieren.]**

2. **[P1 – Detailed Design] Vollständige Methodenliste mit Vor-/Nachbedingungen.**
   Alle privaten Methoden der Klasse `EmergencyStop` mit Preconditions, Postconditions und ggf. Invarianten spezifizieren (Design by Contract). **[TODO: Tabelle mit Pre-/Postconditions für alle privaten Methoden vervollständigen.]**

3. **[P2 – Architektur] Kurze schriftliche Begründung der Sprachauswahl (C++) finalisieren.**
   Der Abschnitt 6.1 enthält bereits die wesentlichen Argumente. Mit dem Team abstimmen und auf 3–5 Sätze verdichten. **[TODO: Sprachauswahl-Begründung mit Team abstimmen und finalisieren.]**

4. **[P2 – Architektur] CAN-Datenklasse ausdetaillieren.**
   Die Klasse für CAN-Pakete (Struktur: ID, Payload, Timestamp) in einem eigenen Klassendiagramm modellieren und in Abschnitt 2.3 einbinden. **[TODO: CAN-Datenklasse modellieren und Diagramm einbinden.]**

5. **[P3 – Qualität] Latenzzahlen ergänzen.**
   Typische CAN-Zykluszeit und LAN-Latenz aus Systemtests eintragen (Abschnitt 3). **[TODO: Messwerte aus Systemtest nachtragen.]**

6. **[P3 – Optional] Logger-Interface vollständig spezifizieren.**
   Vollständige Methodenliste für `Logger` (inkl. Log-Level-Enum, Puffer-Strategie, Ausgabe-Kanal). **[TODO: Logger-Interface ausarbeiten (optional, aber gern gesehen laut Aufgabenstellung).]**

7. **[P3 – Optional] Interfaces (`EnvironmentControl`) vollständig ausarbeiten.**
   Nicht nur eine einzelne Funktion, sondern alle Methoden und Signale des Interfaces spezifizieren. **[TODO: EnvironmentControl-Interface vollständig modellieren.]**

8.2 Bekannte Annahmen (zur Überprüfung)

Die folgenden Annahmen wurden getroffen und müssen mit dem Team bzw. der Aufgabenstellung validiert werden:

- *[Annahme: Emergency Stop läuft auf dem Arduino Mega Slave (nahe DriveControlUnit). Erste Vermutung laut Mitschrieb; zu bestätigen.]*
- *[Annahme: CAN-Zykluszeit* $< 5\,ms$*; zu messen.]*
- *[Annahme: Distanzschwelle für Rot-Zone:* $\leq 15\,\mathrm{cm}$*.]*
- *[Annahme: CAN-Datenklasse kapselt ID, Payload und Timestamp.]*
- *[Annahme: Logger schreibt auf seriellen Ausgang des Slave-Arduinos; kein persistenter Speicher.]*